

dcc CIENCIAS DE LA COMPUTACIÓN
UNIVERSIDAD DE CHILE

MINERÍA DE DATOS

Clase: Redes Neuronales

Profesores: Cinthia Sánchez y Cristian Llull

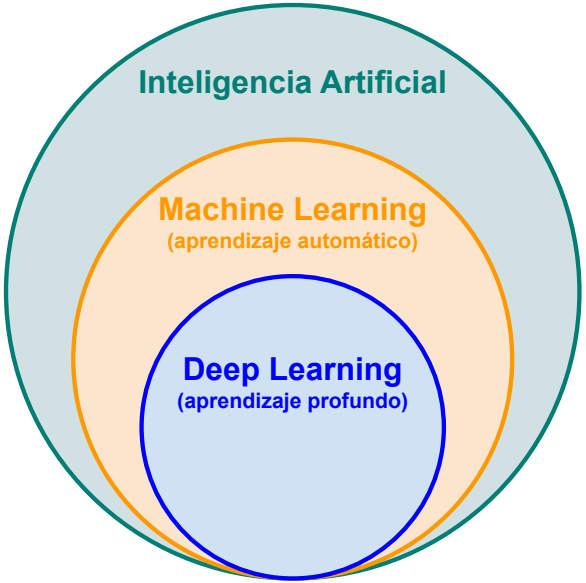
Contenidos

- Deep Learning
- Neurona y redes neuronales
- Componentes
- Entrenamiento
- Generalización
- Representaciones y arquitecturas

“One of the central challenges of computer science is to get a computer to do what needs to be done, without telling it how to do it.”

John Koza

Deep Learning

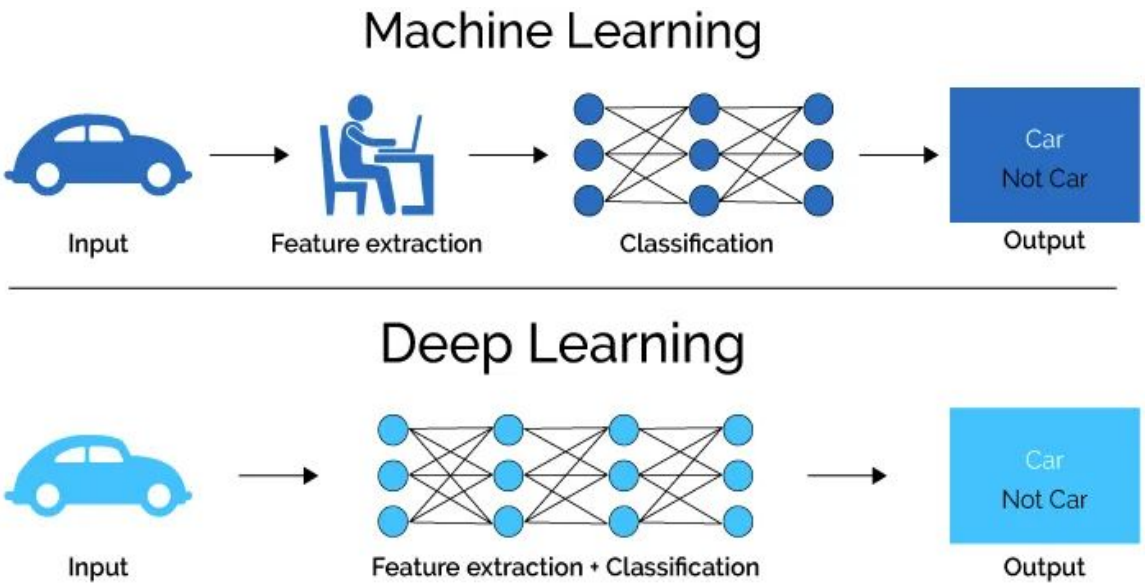


El **Aprendizaje Profundo** ha tenido un importante rol en el desarrollo de la IA en la última década.

Se basa en las **redes neuronales artificiales**, modelos altamente expresivos y generales que han sido capaces de resolver problemas de variada índole.

Las redes neuronales son **modelos de aprendizaje automático inspirados en el cerebro** muy populares formados por unidades llamadas **neuronas**.

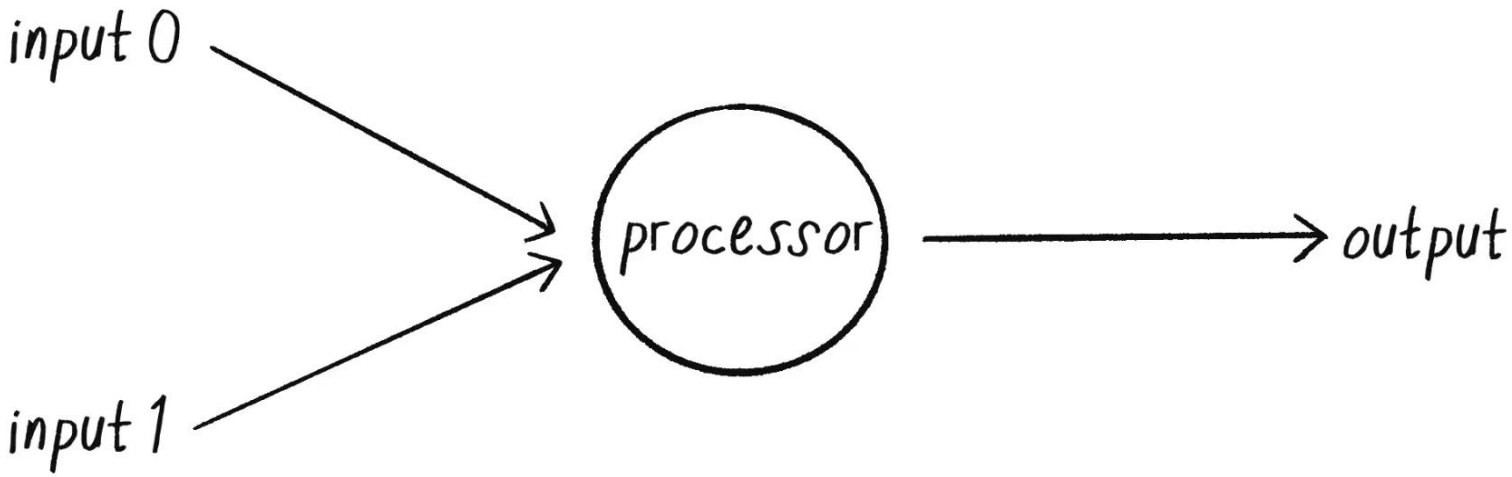
¿Cuál es la diferencia entre ML y DL?



https://lawtomated.com/a-i-technical-machine-vs-deep-learning/

**Como se mencionó, DL es un subconjunto de ML. Esta pregunta hace referencia a ML sin DL vs DL.*

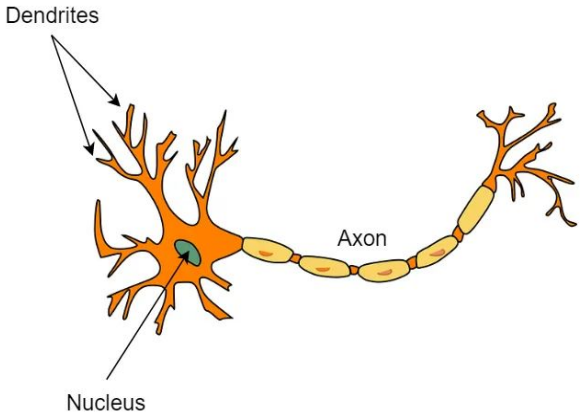
Neurona artificial: Perceptrón

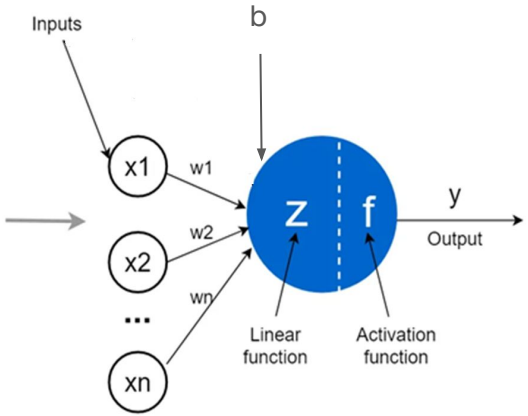


Inventado en 1957 por Frank Rosenblatt, Cornell Aeronautical Laboratory

Neurona artificial: Perceptrón

- Simplificación de una neurona (natural) que representa la activación de una neurona como una combinación lineal de los estímulos recibidos.
- Una neurona es una unidad computacional que tiene entradas y salidas escalares.
- Cada entrada tiene un peso asociado ***w***.
- La neurona multiplica cada entrada por su peso y luego las suma.
- Luego aplica una función de activación **g** (generalmente no lineal) al resultado, y la pasa a su salida.





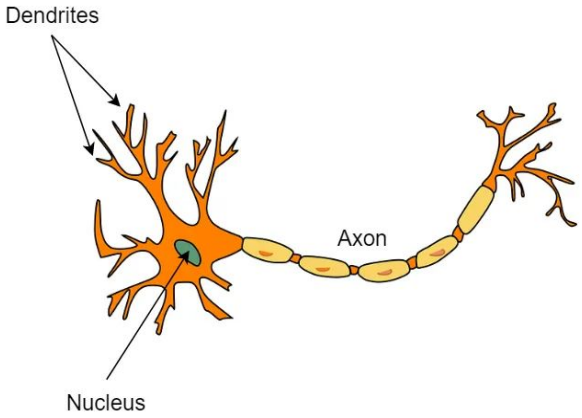
$$z = x_1 w_1 + x_2 w_2 + \ldots + x_n w_n + b$$

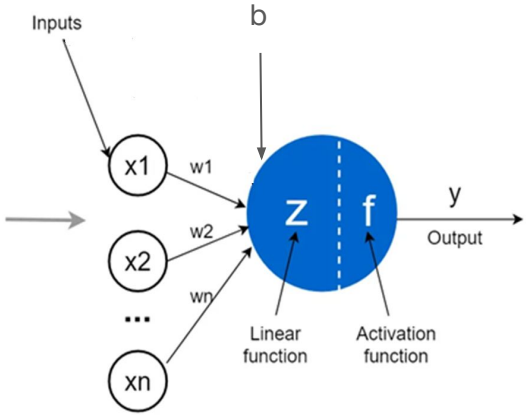
$$y = f(z)$$

https://towardsdatascience.com/

Neurona artificial: Perceptrón

- Simplificación de una neurona (natural) que representa la activación de una neurona como una combinación lineal de los estímulos recibidos.
- Una neurona es una unidad computacional que tiene entradas y salidas escalares.
- Cada entrada tiene un peso asociado ***w***.
- La neurona multiplica cada entrada por su peso y luego las suma.
- Luego aplica una función de activación **g** (generalmente no lineal) al resultado, y la pasa a su salida.





$$z = x_1 w_1 + x_2 w_2 + \ldots + x_n w_n + b$$

$$y = f(z)$$

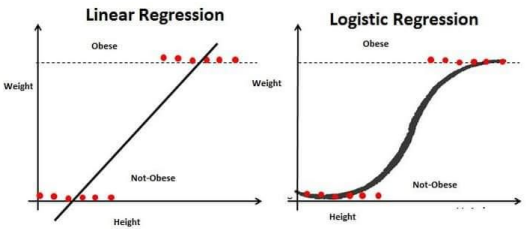
https://towardsdatascience.com/

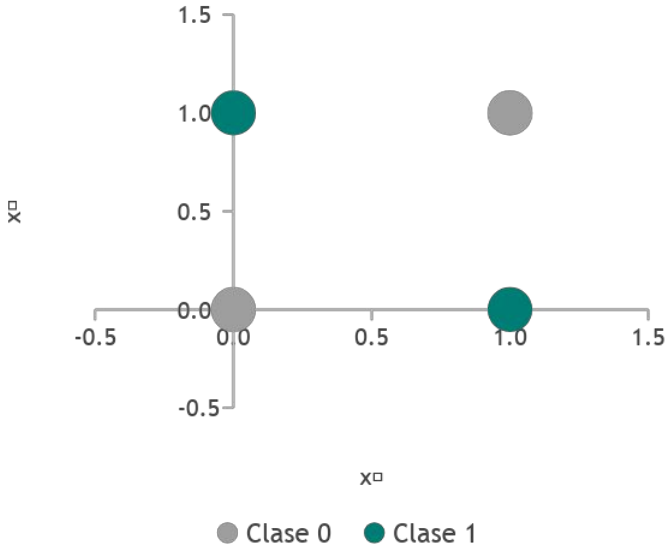
Perceptrón y modelos lineales

- Con activación identidad, una neurona es una **regresión lineal**.
- Con activación sigmoide, es una **regresión logística**.
- Su frontera de decisión es un hiperplano: no puede separar clases como las de XOR.

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b \qquad \hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

Regresión lineal **Regresión logística**



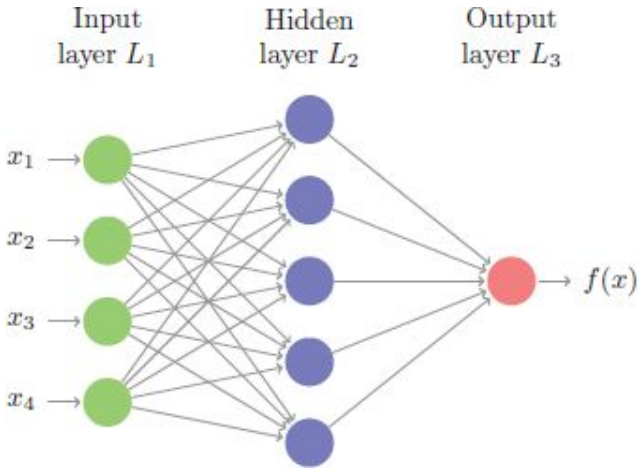


XOR: ninguna recta separa las dos clases

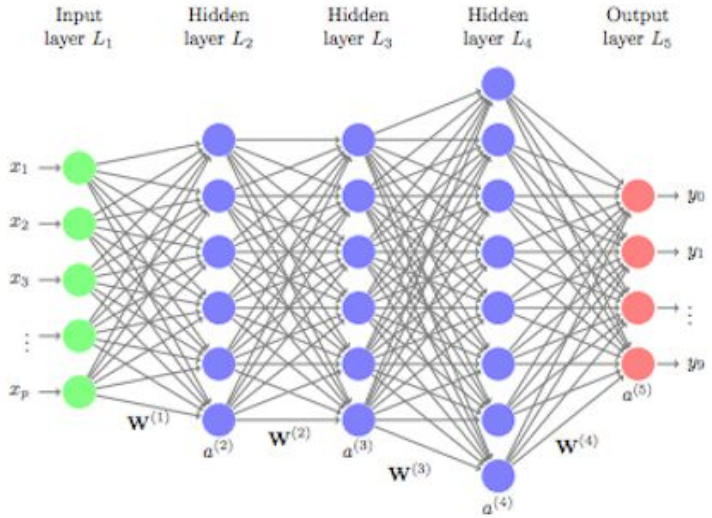
Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Redes neuronales artificiales

- Modelos de aprendizaje automático formados por unidades llamadas **neuronas**, capaces de aprender relaciones no-lineales entre **x** e **y**.
- Varias neuronas pueden agruparse en una capa de neuronas.
- La salida de una capa puede pasarse como input a otra capa de forma de cascada.



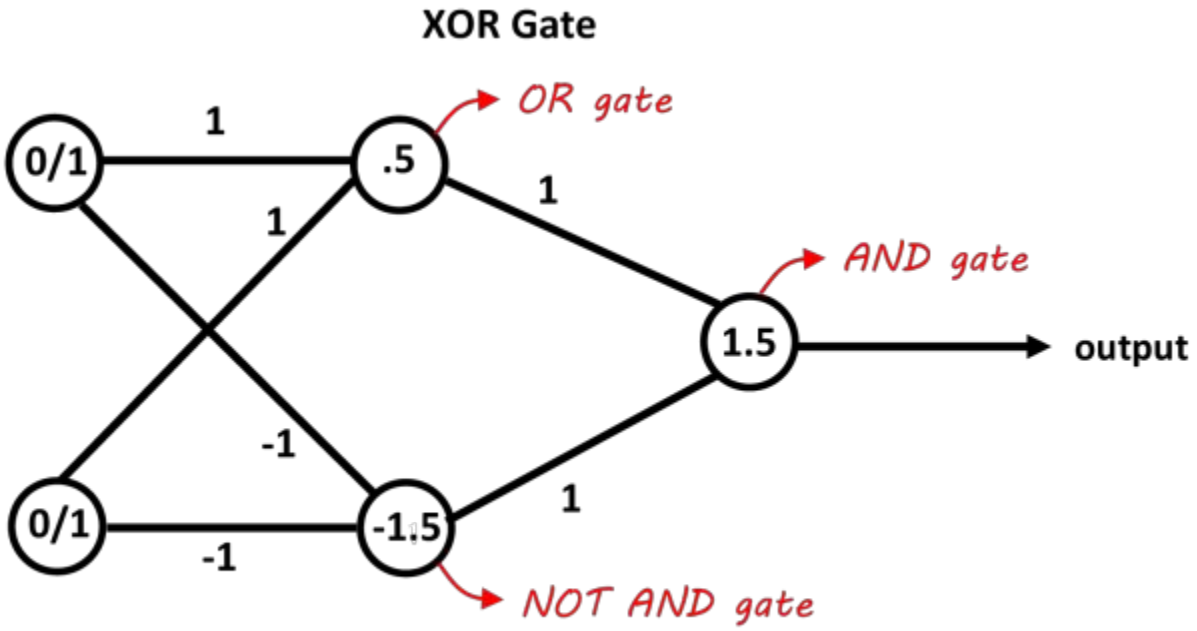
Red Neuronal de una capa oculta



Red Neuronal profunda

https://afit-r.github.io/feedforward_DNN

XOR Gate


XOR Gate
OR gate
AND gate
NOT AND gate
0/1
0/1
.5
-1.5
1.5
output
1
1
1
-1
-1
1

Propagación hacia adelante (forward pass)

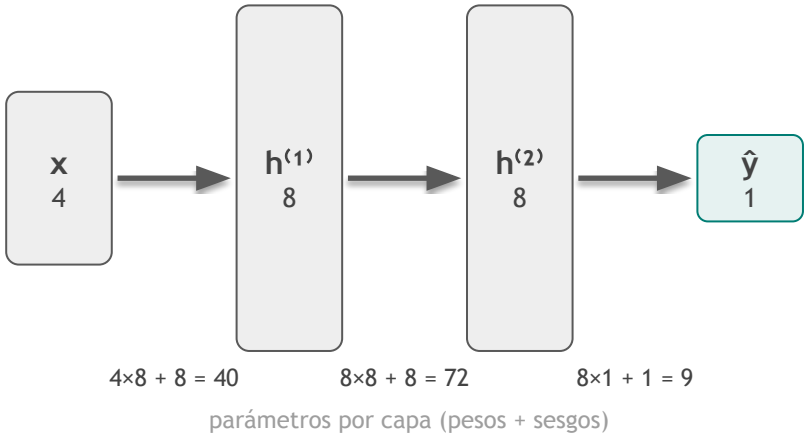
$$\mathbf{z}^{(\ell)} = W^{(\ell)}\,\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$$

$$\mathbf{a}^{(\ell)} = g(\mathbf{z}^{(\ell)})$$

Cada capa aplica una transformación lineal y luego una activación: como una regresión logística, pero con muchas salidas a la vez.

$$\hat{y} = g^{(L)}(W^{(L)} \cdots g^{(1)}(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \cdots + \mathbf{b}^{(L)})$$

La red completa es una **composición de funciones**.



Total: 121 parámetros.
Una regresión logística con 4 entradas tiene 5.

Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

¿Por qué funciones de activación no lineales?

Sin activación no lineal, apilar capas no agrega nada:

$$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\,\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$$

→ el resultado sigue siendo una **función lineal** de x, sin importar cuántas capas haya.

Con una capa oculta no lineal
la red ya resuelve XOR: transforma el espacio para que las clases queden separables.

Teorema de aproximación universal
Una capa oculta con suficientes neuronas puede aproximar cualquier función continua tanto como se quiera.

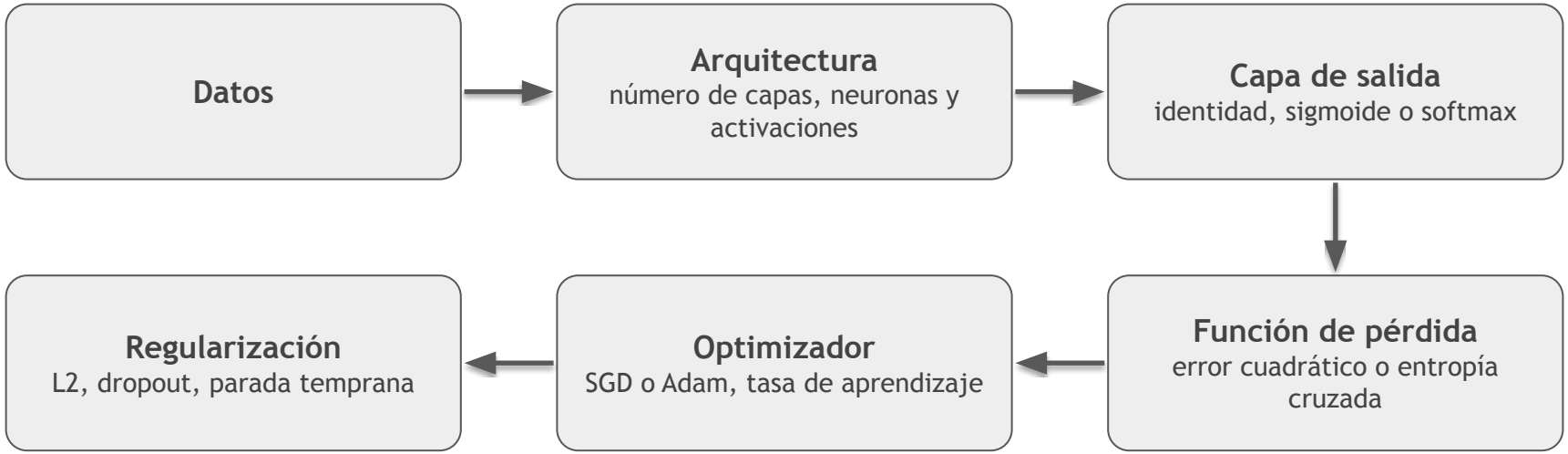
Pero el teorema no dice cuántas neuronas se necesitan, ni garantiza que el entrenamiento encuentre esos pesos, ni que el modelo generalice.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals and Systems.
Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Redes neuronales artificiales

- Las redes neuronales son modelos muy poderosos para regresión y clasificación.
- El uso de redes con varias capas se llama popularmente como **Deep Learning**.
- Existen arquitecturas de red que son buenas para aprender representaciones sobre datos complejos: texto, imágenes, audio, video.
- Arquitecturas famosas: redes neuronales convolucionales, redes recurrentes y redes de atención (transformers).
- La alta capacidad de estas redes las hace muy proclives a sobreajuste (*overfitting*). Hay varias técnicas para mitigarlo: regularización, drop-out, batch normalization.

Componentes de una red neuronal

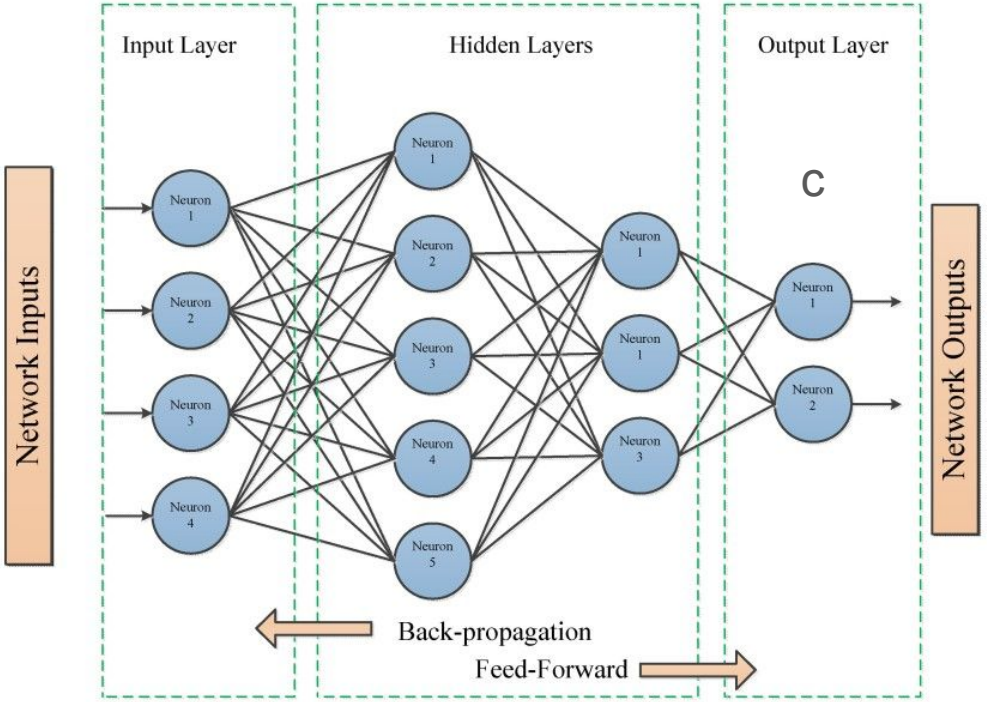


Todas son decisiones de diseño (**hiperparámetros**): se eligen con validación, nunca con el conjunto de test (clase 8).

Descenso de Gradiente para encontrar los parámetros

Dados los datos ($D = \{(x^1, y^1), (x^2, y^2), \ldots, (x^n, y^n)\}$ y los parámetros de la red $\theta = (W^1, b^1, \ldots, W^L, b^L, c)$





https://www.linkedin.com/pulse/feedforward-vs-backpropagation-ann-saffronedge1/

$\hat{y}^i$ y^i = gato

$$L = \frac{1}{n} \sum_{i=1}^{n} \text{error}(\hat{y}^i, y^i)$$

Error (o loss)

Capa de salida y función de pérdida

Tarea	Activación de salida	Función de pérdida
Regresión	Identidad: $\hat{y} = z$	Error cuadrático medio (MSE)
Clasificación binaria	Sigmoide: $\hat{y} \in (0, 1)$	Entropía cruzada binaria
Clasificación multiclase	Softmax: probabilidades que suman 1	Entropía cruzada categórica

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad L = -\sum_k y_k \log \hat{y}_k$$

y_k = 1 para la clase correcta y 0 para las demás: la pérdida es −log de la probabilidad asignada a la clase correcta.

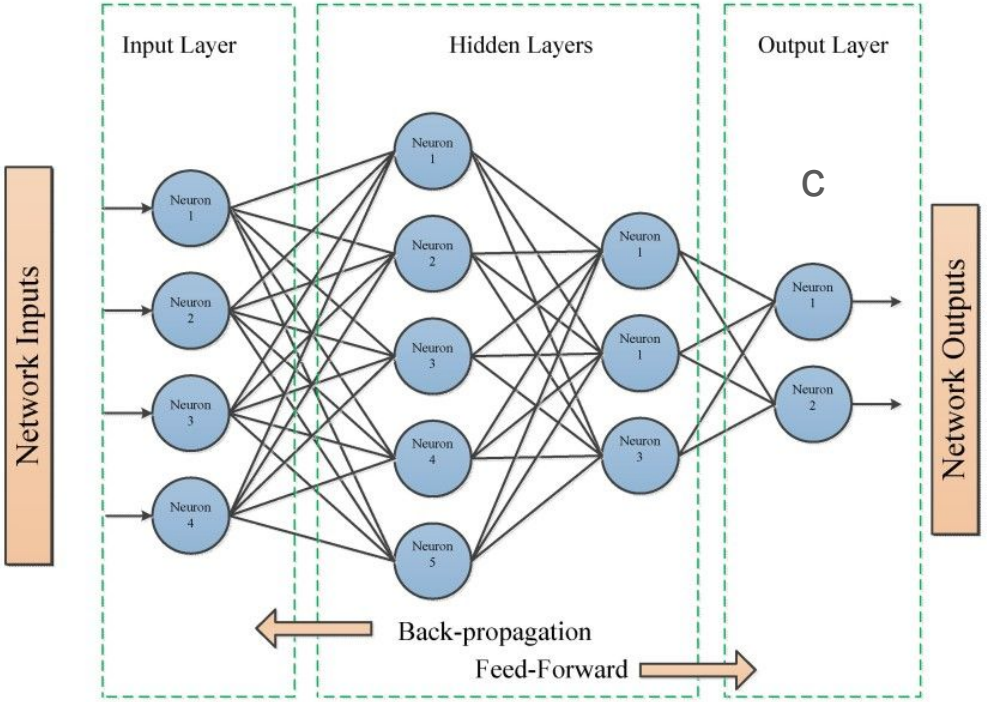
Ejemplo
Puntajes (2,0; 1,0; 0,1) → softmax (0,66; 0,24; 0,10).
Si la clase correcta es la 1ª: L = −ln 0,66 ≈ 0,42.
Si solo le diera 0,10: L = −ln 0,10 ≈ 2,30.
Castiga los errores cometidos con confianza.

Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Descenso de Gradiente para encontrar los parámetros

Dados los datos ($D = \{(x^1, y^1), (x^2, y^2), \ldots, (x^n, y^n)\}$ y los parámetros de la red $\theta = (W^1, b^1, \ldots, W^L, b^L, c)$





$$L = \frac{1}{n} \sum_{i=1}^{n} \text{error}(\hat{y}^i, y^i)$$

$$\underset{\theta}{\text{argmin}} \; L(\theta)$$

$\hat{y}^i$

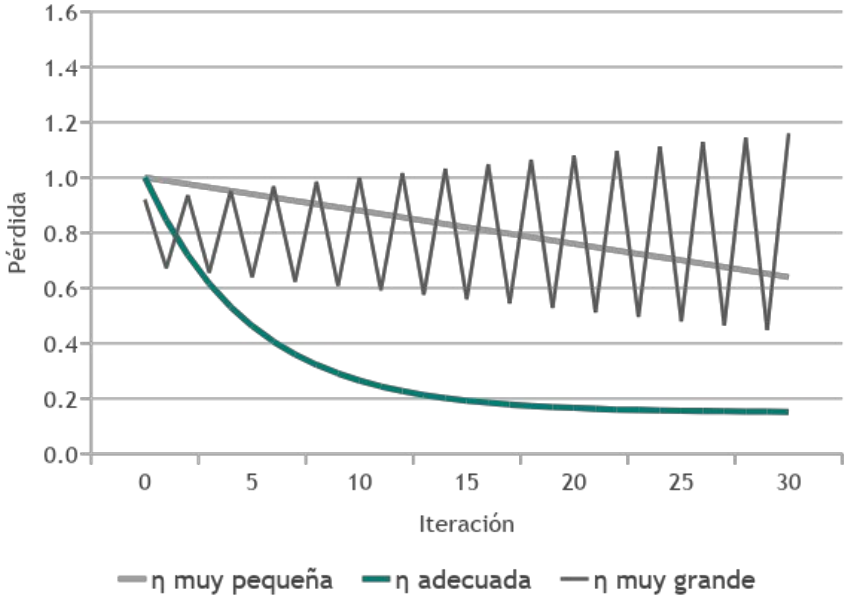
Objetivo de entrenar la red: Encontrar θ que minimiza la función de error.

https://www.linkedin.com/pulse/feedforward-vs-backpropagation-ann-saffronedge1/

Descenso de gradiente y tasa de aprendizaje

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$$

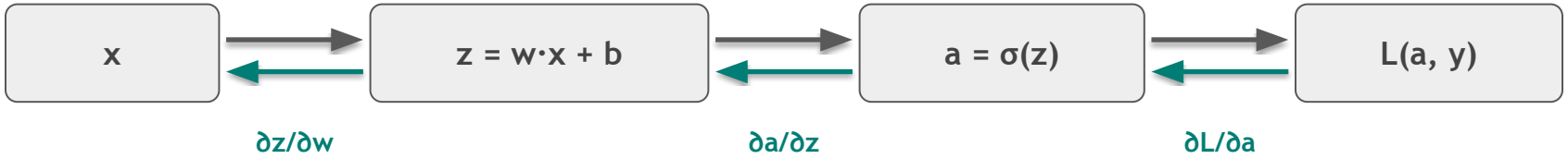
- El gradiente indica hacia dónde crece la pérdida; se avanza en sentido contrario.
- La **tasa de aprendizaje η** controla el tamaño del paso.
- A diferencia de la regresión logística (convexa), la pérdida de una red tiene muchos mínimos locales: el resultado depende de la inicialización aleatoria.



Curvas ilustrativas

Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Retropropagación (backpropagation)



Gris: hacia adelante, se calculan y guardan los valores. Verde: hacia atrás, se multiplican las derivadas locales.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

- Reutiliza los cálculos intermedios: obtener todos los gradientes cuesta del mismo orden que una propagación hacia adelante.
- PyTorch y TensorFlow lo hacen automáticamente (**diferenciación automática**).

Rumelhart, D., Hinton, G. y Williams, R. (1986). Learning representations by back-propagating errors. Nature.
Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Mini-batch, épocas y optimizadores

Iteración
una actualización de los pesos

Batch (lote)
ejemplos usados en una iteración

Época
una pasada completa por el entrenamiento

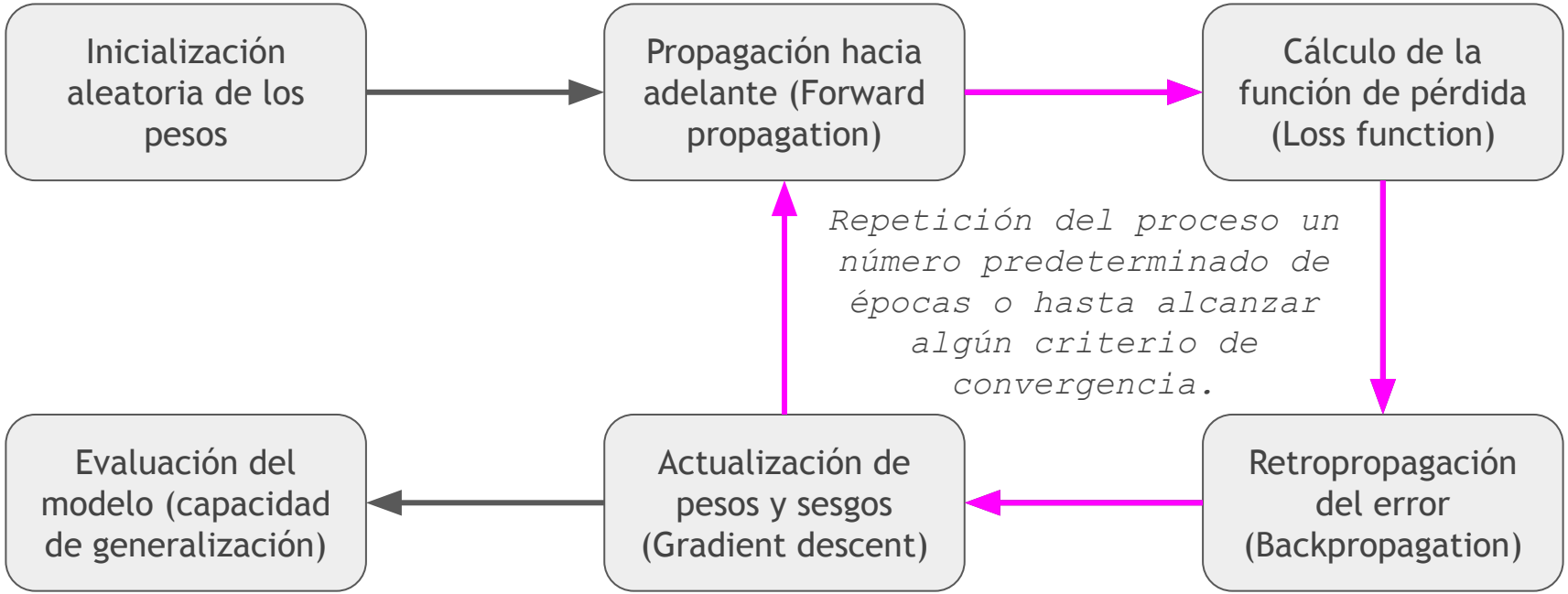
Ejemplo: 10.000 ejemplos y batch de 100 → **100 iteraciones por época**.

Variante	Ejemplos por paso	Gradiente
Batch completo	todos	exacto, pero caro
Mini-batch	decenas o cientos	**equilibrio (lo habitual)**
Estocástico (SGD)	1	muy ruidoso

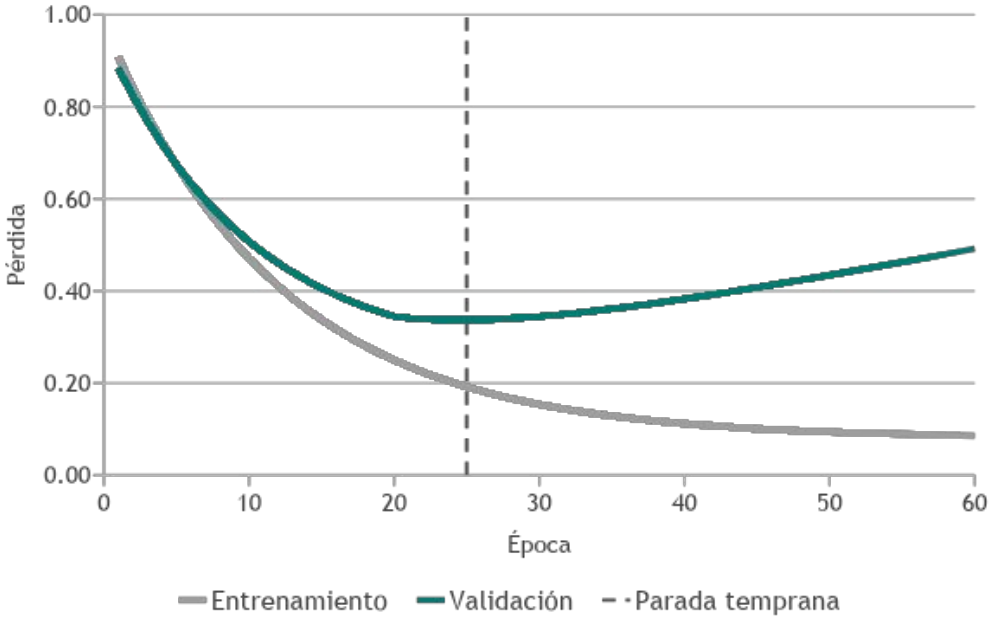
Optimizadores
SGD con momentum: acumula la dirección de los pasos anteriores.
Adam: adapta la tasa de cada parámetro; es el punto de partida habitual. En scikit-learn es el solver por defecto, con $\eta = 0{,}001$.

Kingma, D. y Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.
scikit-learn (s. f.). sklearn.neural_network.MLPClassifier. Documentación oficial.

Entrenamiento y evaluación de una red neuronal



Sobreajuste y curvas de aprendizaje



- Muchos parámetros dan capacidad para memorizar el entrenamiento.
- La pérdida de validación baja, toca un mínimo y vuelve a subir.
- Parada temprana: detenerse en ese mínimo y conservar **los pesos de esa época**.
- Se monitorea con validación, no con test.

Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Regularización en redes neuronales

Penalización L2 (weight decay)
Suma a la pérdida el tamaño de los pesos: $L + \lambda \Sigma w^2$. En scikit-learn es el parámetro alpha.

Dropout
Apaga neuronas al azar en cada paso del entrenamiento, para que no dependan unas de otras.

Parada temprana (early stopping)
Detiene el entrenamiento cuando la pérdida de validación deja de mejorar.

Más datos y aumento de datos
Genera variaciones de los ejemplos, como rotar o recortar imágenes. Lo verán en visión por computador.

La normalización por lotes (batch normalization) se usa sobre todo para estabilizar el entrenamiento.

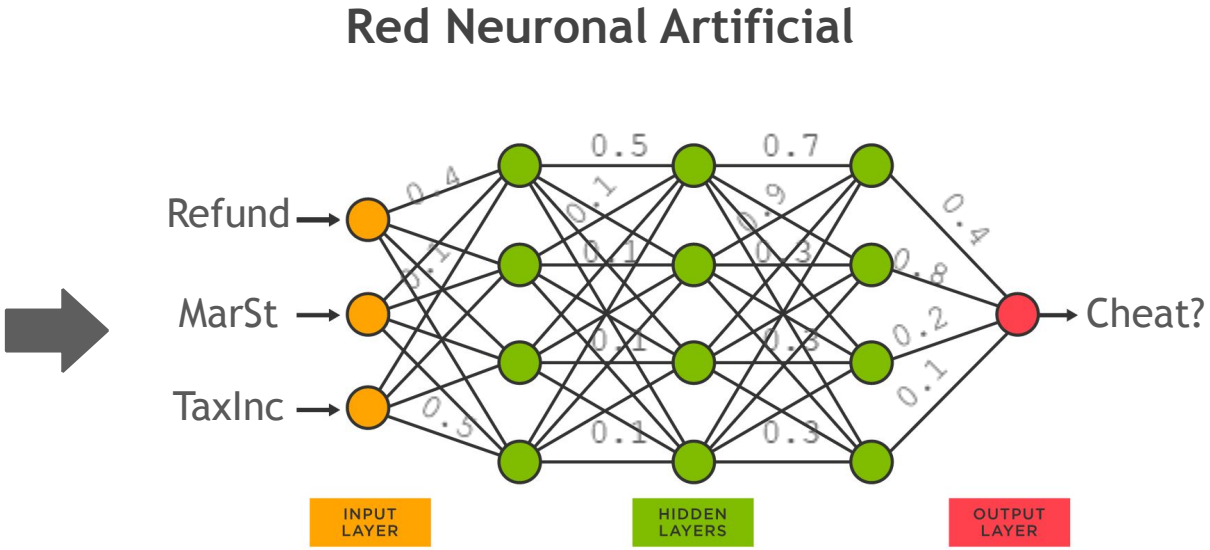
Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR.
Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep Learning. MIT Press.

Redes neuronales artificiales

Aprender de forma automática de los datos. La máquina desarrolla su propia representación.

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Datos de Entrenamiento

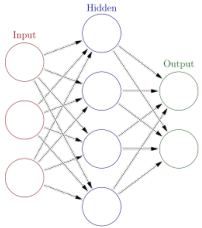


Arquitecturas según el tipo de dato

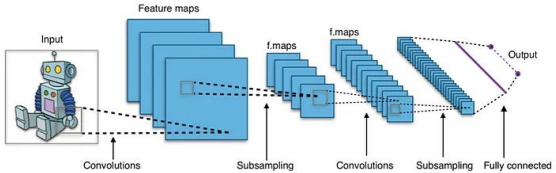
Datos tabulares
Perceptrón multicapa (MLP)
Capas densas: cada neurona conectada a todas las de la capa anterior.
Lo que vimos hoy

Imágenes
Redes convolucionales (CNN)
Filtros que recorren la imagen y detectan patrones locales.
Visión por computador

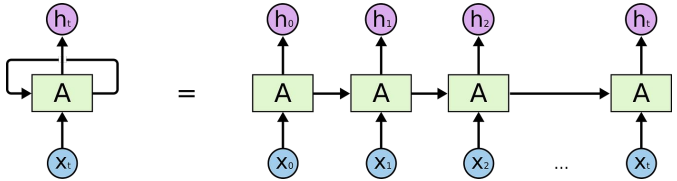
Texto y secuencias
Redes recurrentes (RNN) y Transformers
Procesan secuencias; los Transformers usan mecanismos de atención.
NLP



MLP



CNN

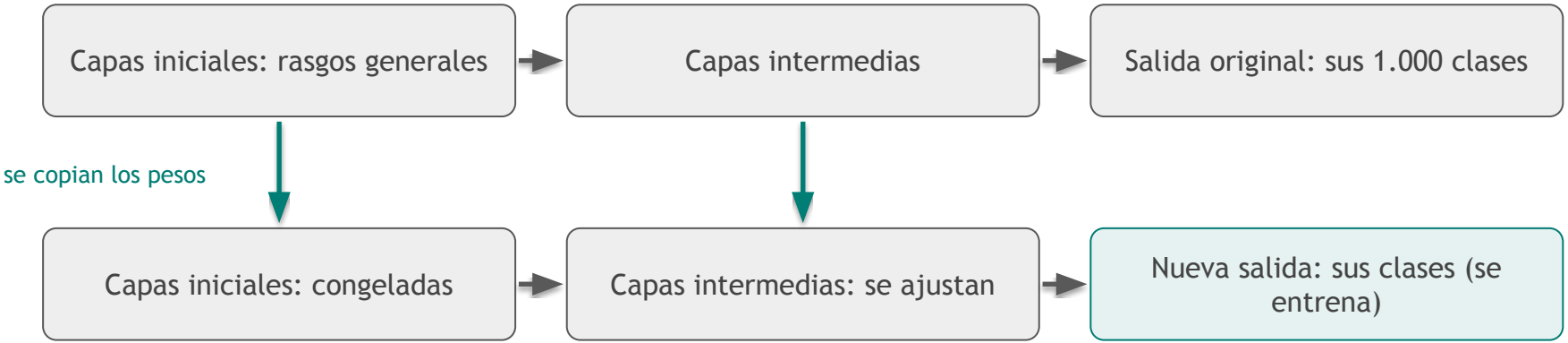


RNN

LeCun, Y. et al. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE.
Hochreiter, S. y Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.

Transfer learning y fine-tuning

Modelo preentrenado con un conjunto grande (por ejemplo, millones de imágenes)



- Las primeras capas aprenden rasgos generales y las últimas, rasgos específicos de la tarea original.
- Permite entrenar con pocos datos. **Pueden encontrar modelos preentrenados en Hugging Face**.

Yosinski, J. et al. (2014). How transferable are features in deep neural networks?. NeurIPS.

Redes neuronales en datos tabulares

45

conjuntos tabulares de tamaño medio (~10.000 ejemplos): los modelos de árboles, como Random Forest y XGBoost, superaron a las redes neuronales (2022).

2025

TabPFN, un modelo fundacional para tablas basado en Transformers, superó a los métodos previos en conjuntos de hasta 10.000 ejemplos.

Las redes destacan con datos no estructurados: imágenes, texto y audio.

Grinsztajn, L., Oyallon, E. y Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data?. NeurIPS.
Hollmann, N. et al. (2025). Accurate predictions on small data with a tabular foundation model. Nature.

Aplicabilidad versus interpretabilidad

- Aumentando el número de capas y de neuronas por capas podemos conseguir estructuras más y más complejas.
- Sin embargo, la red es un modelo general que requiere entrenamiento, es decir, encontrar los parámetros.
- Para redes grandes, encontrar los parámetros es un desafío.
- Los avances recientes en DL han permitido entrenar redes con millones (e incluso billones) de parámetros.

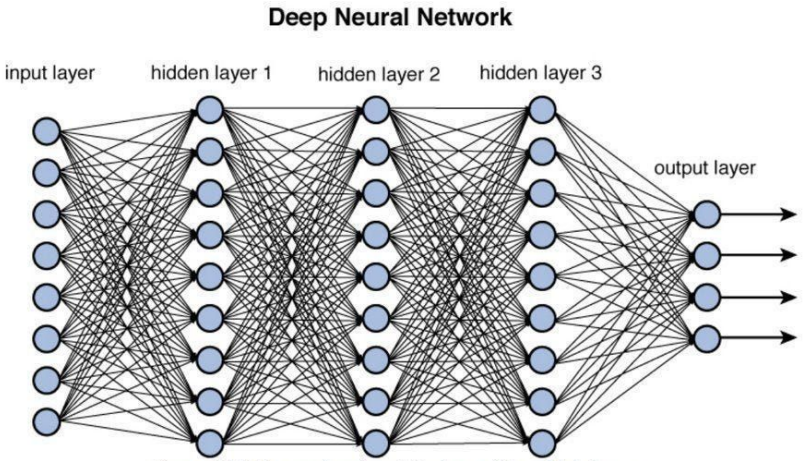


Figure 12.2 Deep network architecture with multiple layers.

https://towardsdatascience.com/training-deep-neural-networks-9fdb1964b964

Implementación en scikit-learn

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier

modelo = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(64, 32),
                  activation="relu",
                  solver="adam",
                  learning_rate_init=0.001,
                  alpha=1e-4,
                  early_stopping=True,
                  random_state=0))
modelo.fit(X_train, y_train)
```

StandardScaler: las redes son sensibles a la escala

(64, 32): dos capas ocultas de 64 y 32 neuronas

alpha: penalización L2

early_stopping: separa un 10% como validación

random_state: la pérdida no es convexa: fijar la semilla

Para redes profundas y GPU se usan PyTorch o Keras (TensorFlow).

scikit-learn (s. f.). sklearn.neural_network.MLPClassifier. Documentación oficial.

En resumen

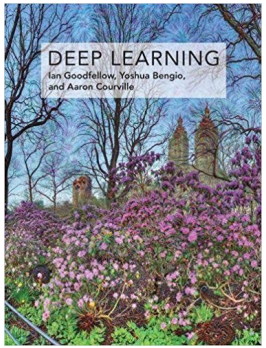
1 Una neurona es un modelo lineal más una activación; una red es una composición de capas no lineales.

2 Entrenar es minimizar una pérdida con descenso de gradiente; backpropagation calcula los gradientes.

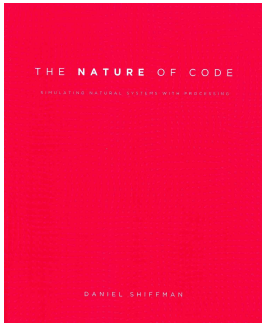
3 Con muchos parámetros sobreajustan fácilmente: validación, regularización y parada temprana.

4 Su fuerza está en aprender representaciones de datos no estructurados: texto, imágenes y audio.

Si quieren involucrarse en el tema…



Deep Learning, by Ian Goodfellow



https://natureofcode.com/neural-networks/, Daniel Shiffman